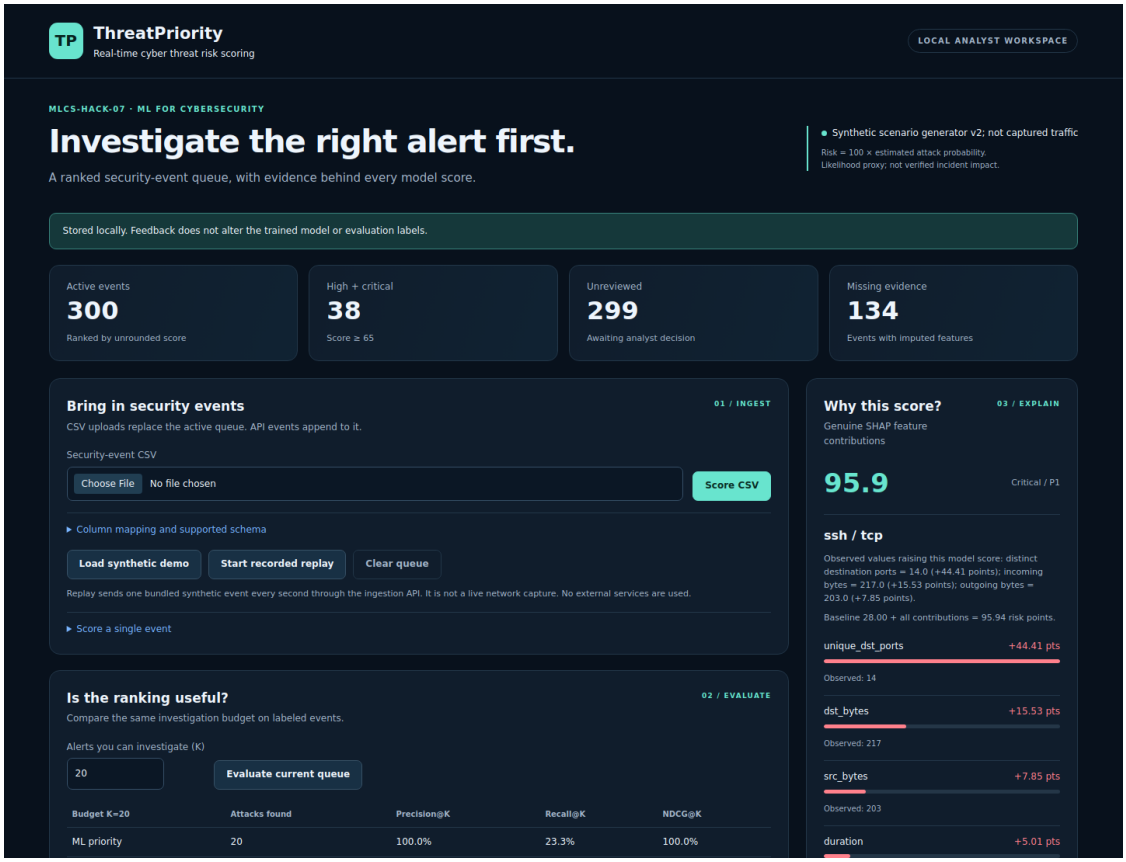
ThreatPriority

Real-Time Cyber Threat Risk Scoring
Project report and validation evidence

Problem: Security analysts receive more events than they can investigate at once. The system assigns incoming events a 0-100 attack-likelihood score, ranks the queue, and explains each prediction so the analyst can choose what to inspect first.

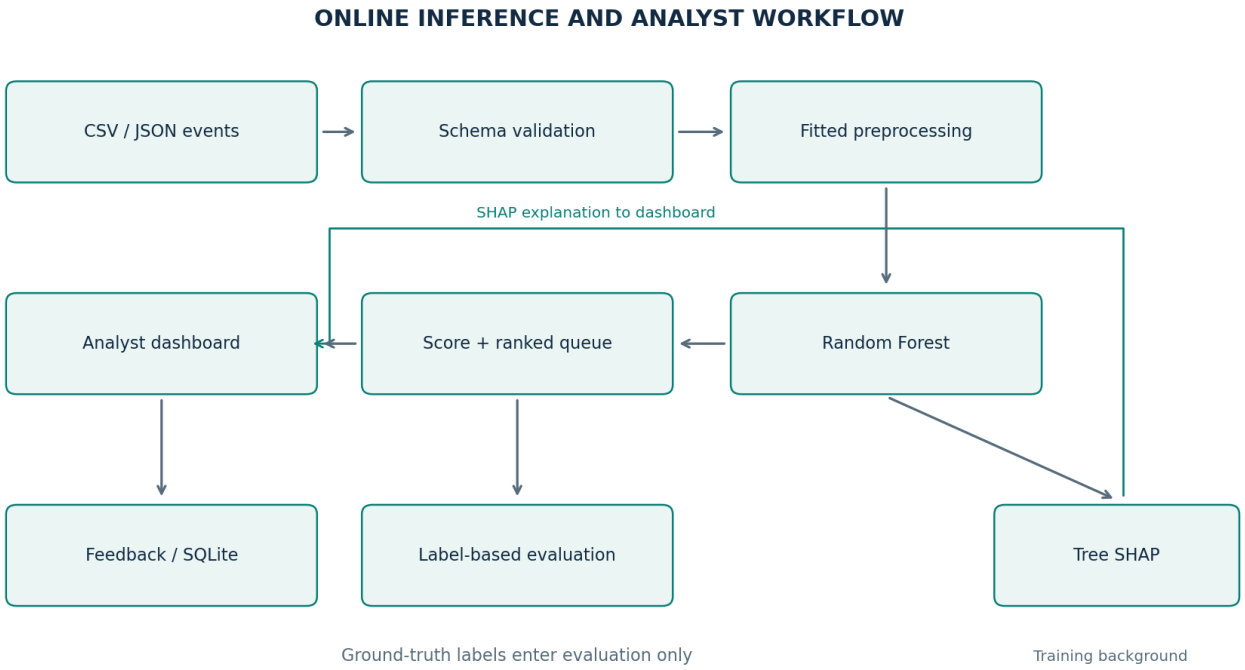
Implementation: Python 3.12, Flask, scikit-learn Random Forest, genuine Tree SHAP, local HTML/CSS/JavaScript dashboard, and SQLite feedback. Code, model, datasets, architecture, tests and jury guide are bundled.



Dashboard excerpt: ingestion, investigation-budget comparison and an event-specific SHAP explanation. The full queue is available by scrolling.

Data disclosure: This release uses 6,000 documented synthetic events. It contains no downloaded Kaggle or production logs. Results validate the simulated distribution; they are not evidence of real-world attack-detection performance. Compatible public datasets require mapping, provenance checks and retraining.

1. Architecture and end-to-end workflow



CSV/JSON events are validated, transformed by training-fitted preprocessing, and scored by Random Forest. The queue sorts unrounded scores. SHAP uses the same transformed event and a training-only background. Labels enter the evaluation branch; feedback remains a separate local record.

Stage	What happens
Offline training	Synthetic generator or compatible labeled CSV; duplicate removal; 60/20/20 split; train-only preprocessing and augmentation; baseline comparison; test evaluation; serialized model.
Online ingestion	CSV upload replaces queue; event API appends records. The dashboard polls every 2 seconds for external incoming events. Replay sends one recorded synthetic event per second.
Explanation	Tree SHAP explains class-1 probability. Encoded contributions are grouped by source feature and checked for additive reconstruction.
Analyst workflow	Inspect top-ranked events, examine missingness and explanations, record a decision, export scores or retrieve evaluation JSON.

Queue state resets on restart. Feedback history is stored in runtime/feedback.sqlite. The local app has no packet capture, external services, automatic blocking, production SOC integration or automatic online retraining.

2. Dataset, labels and security features

The generator includes normal web traffic, backups, password mistakes, busy services, brute force, scans, exfiltration and floods. Benign and attack distributions overlap. Scenarios set binary labels before training; 2.5% label-independent flips model imperfect attribution. Random feature missingness is 7%.

Feature group	Meaning
duration	Connection duration in seconds.
src_bytes / dst_bytes	Directional byte volumes. Large outgoing transfers can be backups or suspicious transfers.
src_packets / dst_packets	Directional packet counts; high counts can reflect busy services or flooding.
failed_logins	Synthetic preceding-five-minute authentication-failure count.
unique_dst_ports	Synthetic preceding-five-minute distinct destination-port count.
protocol / service	Observed network protocol and application service, one-hot encoded.

Window counts are supplied feature summaries; the app does not compute them from captured historical logs. All units and assumptions are documented in DATA_DICTIONARY.md. Missing public-dataset measurements should be omitted rather than fabricated.

Leakage controls: Only nine allowlisted features enter X. Label, scenario, attack family, ID and timestamp are excluded. Feature-identical rows are removed before splitting. Imputation, encoding and SHAP background use only training data. A training-only masked copy augments missingness without creating independent evidence.

Split	Rows	Purpose
Training	3,600	Learn preprocessing and model patterns.
Validation	1,200	Compare Logistic Regression and Random Forest.
Test	1,200	Measure the fixed forest on held-out records.
Demo	300	New seed; same synthetic distribution.

A random split is appropriate for this independent synthetic generator; it does not prove chronological generalization. Public logs may require time, session, host or collection-day splits.

3. Model, scores and missing inputs

Random Forest uses 120 trees, maximum depth 10 and minimum 5 samples per leaf. Logistic Regression is a validation baseline. The forest was chosen for nonlinear interactions and native Tree SHAP; the test set is not used to search for a favorable threshold.

Validation model	Precision	Recall	F1
Random Forest	87.6%	62.8%	73.1%
Logistic Regression	84.7%	43.0%	57.0%

Risk score = 100 x estimated attack probability. The forest output is uncalibrated; it is a likelihood proxy rather than a guaranteed attack percentage or a measured business-impact score. The event category is observed service/protocol, not a predicted attack family.

Score band	Severity / priority
0 to below 35	Low / P4
35 to below 65	Medium / P3
65 to below 85	High / P2
85 to 100	Critical / P1

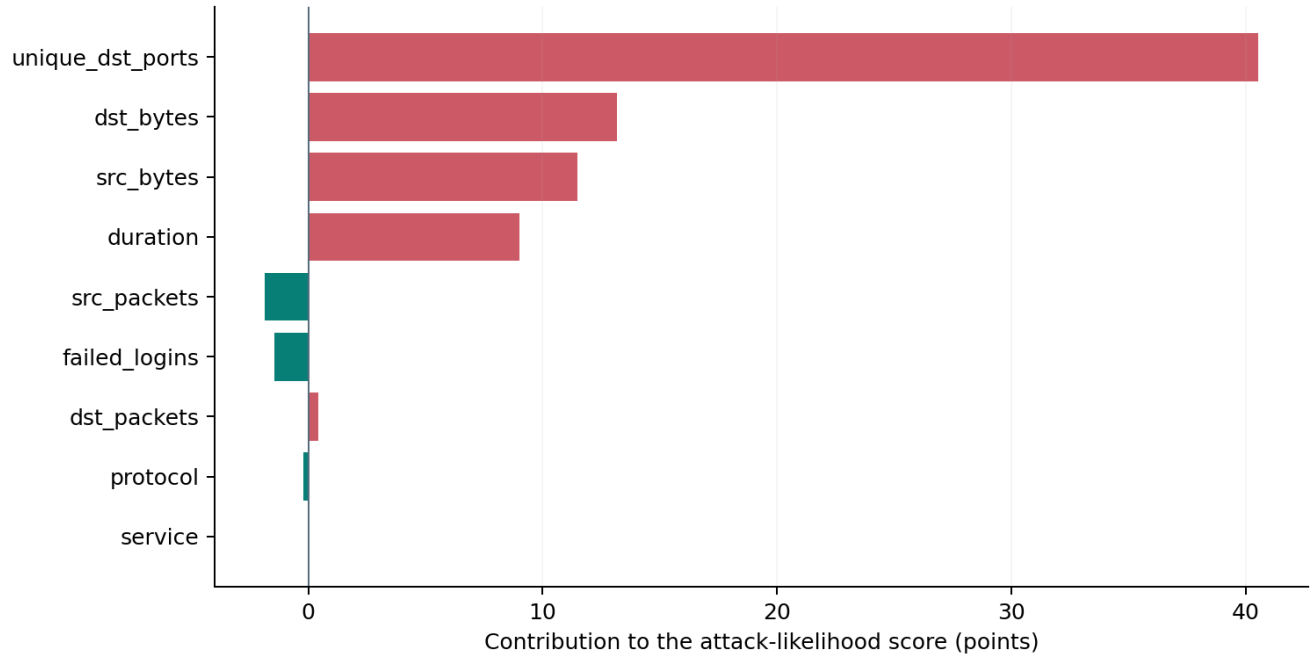
Missing/invalid numeric values use training medians plus fixed missing flags. Missing categorical values receive a dedicated token; unknown categories warn. At least 3 of 9 features must be usable in every row. This is a schema-validity minimum, not a confidence guarantee. Invalid batches are rejected atomically; imputed fields remain visible in the analyst explanation.

No timestamps are invented. ISO strings and Unix seconds are supported. If any time is missing/invalid, the arrival comparison uses input row order for the whole batch. Missing/unknown labels disable evaluation without preventing compatible-event scoring.

Missing-column stress test: Dropping dst_bytes, dst_packets and failed_logins from the same held-out file produced F1 51.5%, FPR 1.6%, and Precision@20 100.0%. This is a fixed-model robustness check; it was not used to tune the model.

4. SHAP: why is this event suspicious?

Tree SHAP is computed in probability space using interventional feature perturbation and 64 transformed training-background events. Positive values raise the attack estimate relative to the background; negative values lower it. One-hot and missing-indicator values are grouped back to the original input field.



Measured example: baseline 28.00 points + signed contributions = score 99.09. Additive reconstruction error: 0.00000650 points. All displayed contributions come from SHAP; no security-rule weights are relabeled as SHAP.

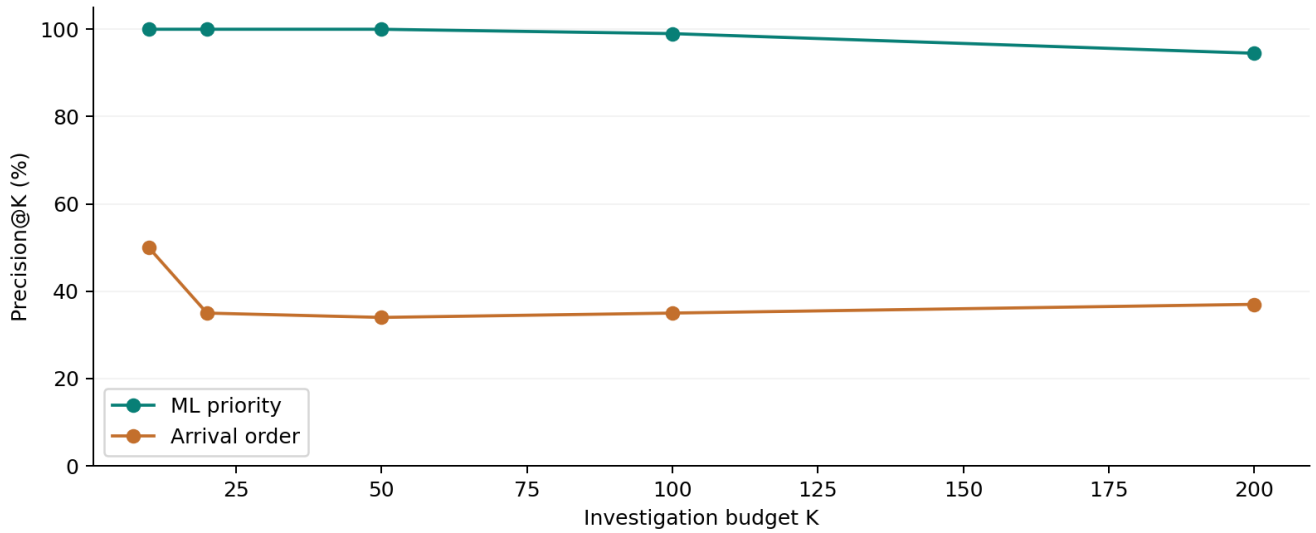
Observed values raising this model score: distinct destination ports = 36.0 (+40.54 points); incoming bytes = 425.0 (+13.16 points); outgoing bytes = 95.0 (+11.50 points).

A missing feature is labeled imputed and is excluded from the short observed-evidence summary. Its contribution can still reflect the imputed value or missing flag. SHAP explains model behavior, not proof of an attack or the causal effect of changing a measurement. Correlated features may divide attribution, and the background selection matters.

Timing: measured warm single-event mean 40.2 ms and p95 57.4 ms over 20 runs. Example SHAP computation 54.4 ms. These Linux-machine measurements are separate from browser refresh delay and are not guaranteed production throughput.

5. Detection and ranking evaluation

Held-out test metric	Measured value
Precision / Recall	90.1% / 62.5%
F1 / False-positive rate	73.8% / 2.8%
Average precision / Brier score	84.0% / 0.1066
Confusion counts	TP 218; FP 24; TN 827; FN 131
Top 20 attacks: ML / arrival	20 / 7
Recall@20: ML / arrival	5.7% / 2.0%



Precision@K = attacks in the top K / K; Recall@K = attacks in the top K / all attacks. Binary NDCG@K discounts lower ranks and normalizes against the best possible placement. The same K and labels are used for ML and arrival queues; ties preserve input order.

Detection metrics use a fixed score threshold of 50. A high top-20 precision can coexist with modest overall recall: 20 investigations cannot recover all 349 test attacks. Undefined quantities, such as Recall@K with no attacks or FPR with no benign events, are shown as N/A. Uploaded-file results describe that file and are not automatically an independent test.

6. Run, validate and explain to the jury

Windows Python 3.12.10: Extract the folder, open it in VS Code, and use Command Prompt. The direct environment executable avoids activation-policy issues. After setup, only the app.py command is needed.

```
py -3.12 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe app.py --port 5050
```

Mac: Use native Python 3.12 (preferably 3.12.10), create the environment and run:

```
python3.12 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python app.py --port 5050
```

Open **http://127.0.0.1:5050**. Install dependencies once with internet; afterward the local app works offline. Setup/run scripts, troubleshooting, the full feature schema and a Kaggle adaptation guide are supplied.

Demo: load demo; compare top-K queues; inspect SHAP; upload missing-column CSV; upload unlabeled CSV; start recorded replay; record feedback; export ranked results. For code questions, open Scorer.score/explain, train.make_preprocessor/train, and evaluation.evaluate. JURY_GUIDE.md contains a presentation script and detailed answers.

Verification: 38 automated tests passed under clean Linux Python 3.12.10; 13 browser checks passed. Binary dependency resolution succeeded for Windows x64 and Apple Silicon macOS Python 3.12. Native Windows/Mac execution was not available here. See VALIDATION.md for exact scope.

References: SHAP TreeExplainer official documentation: shap.readthedocs.io/en/latest/generated/shap.TreeExplainer.html; scikit-learn SimpleImputer documentation: scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html; Lundberg and Lee, A Unified Approach to Interpreting Model Predictions (2017), arxiv.org/abs/1705.07874. Challenge requirements supplied by the user.